

The bundled Wind-AE steady-wind solver: algorithm, continuation, and boundary/initial conditions

EXHALE bundles a 1-D, steady-state, transonic Parker-wind solver — a Fortran port of Wind-AE (Murray-Clay et al. 2009; Broome et al. 2025) — under `src/modules/wind_ae/`. It is used to manufacture a warm-start initial condition for the time-dependent EXHALE run (the IC mode: `windae` key, and the standalone `wind_ae_ic.x`). This note documents what the solver computes, how it solves it (a boundary-value relaxation from the base to the sonic point, followed by an outward initial-value integration), how the boundary and initial conditions are set, and how an arbitrary planet is reached from a shipped seed by parameter continuation. It also records the structure of the Fortran port and the gates it was validated against. Practical usage is in the user manual (§“Wind-AE warm-start initial conditions”); the EXHALE-side BC/IC reference is `EXHALE_BC_and_IC.tex`; the code-by-code comparison with EXHALE and the other escape codes is `code_comparison.tex`; the full porting log is `md_backup/wind_ae_fortran_port_plan.md`.

Contents

1. What the solver computes	1
2. Solution method	2
2.1 Relaxation on the base→sonic interval (the BVP)	2
2.2 Outward integration (the IVP)	2
2.3 The sonic point and its linearization	2
3. Boundary conditions	3
4. Initial condition (the seed) and continuation	3
5. The Fortran port	3
6. Using it in EXHALE	4
7. Scope and limitations	5

1. What the solver computes

Wind-AE solves the *steady-state*, spherically-symmetric, single-streamline (substellar) hydrodynamic equations for an XUV-irradiated H/He atmosphere, from a deep base out through the sonic point to a few planetary radii. The dependent variables are the velocity v , density ρ , temperature T , the neutral fraction Y_s of each species ($s = \text{HI}, \text{He I}$), and the photoionization column density N_{col} of each species. Unlike EXHALE — a time-dependent finite-volume code relaxed to a steady state — Wind-AE solves the stationary equations *directly* as a two-point boundary-value problem (BVP), so there is no time integration and no Courant limit; the cost is that it bakes in the transonic topology (it cannot represent a non-launching or genuinely time-dependent atmosphere).

The governing relations (code units: r in R_{pl} , v in $c_{s,0}$, ρ in ρ_0 , T in $T_0 = 10^4$ K) are mass continuity, the transonic momentum (Parker) equation, energy balance with photoionization heating and line/recombination/Ly α /bolometric cooling, and photoionization–recombination equilibrium for each species:

$$\frac{d\rho}{dr} = -\rho \left(\frac{2}{r} + \frac{1}{v} \frac{dv}{dr} \right), \quad (1)$$

$$\frac{dv}{dr} = \frac{v}{v^2 - \gamma T / \mu} \left[\frac{2\gamma T}{\mu r} - (\gamma - 1) \frac{Q}{v} - \frac{d\phi}{dr} \right], \quad (2)$$

$$\frac{dT}{dr} = (\gamma - 1) \left(\frac{Q\mu}{v} + \frac{T}{\rho} \frac{d\rho}{dr} \right) + \frac{T}{\mu} \frac{d\mu}{dr}, \quad (3)$$

$$\frac{dY_s}{dr} = \frac{\alpha_{\text{rec},s} n_e n_{\text{ion},s} - \Gamma_s}{v n_{\text{tot},s}} \frac{R_{\text{pl}}}{c_{s,0}}, \quad \frac{dN_{\text{col},s}}{dr} = -Y_s n_s \frac{R_{\text{pl}}}{N_{\text{col},0}}, \quad (4)$$

where Q is the net specific heating, ϕ the (optionally tidal) potential, μ the mean molecular weight, $\gamma = 5/3$, Γ_s the photoionization rate (primary + secondary), and $\alpha_{\text{rec},s}$ the recombination coefficient. Equation (2) is singular at the sonic point ($v^2 = \gamma T / \mu$); the solver handles it with an L'Hôpital linearization (§2.3).

2. Solution method

2.1 Relaxation on the base→sonic interval (the BVP)

The interval from the base R_{min} to the sonic point r_s is mapped to a fixed parameter $q \in [0, 1]$ via $r = R_{\text{min}} + qz$, where $z \equiv r_s - R_{\text{min}}$ is itself an unknown (a relaxed variable, constant in r). On a grid of $M = 1501$ points the $N_e = 4 + 2N_{\text{spec}} = 8$ finite-difference residual equations (one per variable: v , z , ρ , T , Y_s , $N_{\text{col},s}$) are solved by the Numerical Recipes `solvd` Newton relaxation. Each iteration assembles the block-tridiagonal Jacobian s by *numerical* differencing of the residuals (`difeq + get_eqn_evals`, step $\delta = x/10^7$), factorizes it (`pinvs/red/bksub`), and updates the solution; it converges quadratically (the shipped seed relaxes to $\|\text{err}\| \sim 10^{-11}$ in ~ 10 iterations). The variable ordering is permuted by `indexv` so the interior boundary conditions come first, as Numerical Recipes requires.

2.2 Outward integration (the IVP)

Past the sonic point the problem is no longer a BVP: with r_s and the state there in hand, the same ODEs (2) are integrated outward to R_{max} as an initial-value problem with an adaptive Bulirsch–Stoer integrator (`odeint+bsstep`, with `mmid/pzextr`); ~ 1000 output points are stored. The near-sonic singularity is past, so dv/dr is evaluated directly.

2.3 The sonic point and its linearization

A transonic wind must pass smoothly through $v = c_s$, where the numerator and denominator of (2) vanish together. Two boundary conditions are imposed at r_s (the last grid point): the *Bernoulli* (numerator) condition and the Mach-1 (denominator) condition $v = c_s \cdot \text{breezeparam}$ (`breezeparam` = 1 for a true transonic wind; < 1 gives a breeze). Approaching r_s , dv/dr is computed by an error-function-weighted blend of the analytic expression and an L'Hôpital linearization ($a(dv/dr)^2 + b(dv/dr) + c = 0$, the positive root = wind), so the derivative stays finite and the relaxation does not stall on the 0/0.

3. Boundary conditions

Base ($r = R_{\min}$, **four conditions**). Density, temperature, and the neutral fraction of each species are pinned to base values: $\rho(R_{\min}) = \rho_{bc}$, $T(R_{\min}) = T_{bc}$, $Y_s(R_{\min}) = 1$ (neutral base). $R_{\min}$ itself, with ρ_{bc} and T_{bc} , follows from a deep, nearly-hydrostatic, bolometrically heated base: the skin/effective temperature from $F_{\text{opt}} = L_*/(4\pi a^2)$, a slant-path $\tau = 1$ surface for ρ , and a microbar base pressure (base_bcs in the reference wrapper).

Sonic point ($r = r_s$, **two conditions**). The Bernoulli and Mach-1 critical-point conditions of §2.3, plus a fixed sonic-point column density $N_{\text{col},s}$ per species (self-consistently the integral of the neutral density from r_s outward).

4. Initial condition (the seed) and continuation

Newton relaxation needs a nearby starting guess, and the equations admit infinitely many breeze solutions near the transonic one; a generic guess relaxes to a breeze, not the wind. Wind-AE therefore *bootstraps* from a single shipped converged solution and *continues* it to the target planet:

- **Seed.** One converged windsoln (the Murray-Clay fiducial hot Jupiter) is shipped (inputdata/windae_seed.csv). It provides both the relaxation guess and the base BCs.
- **Adaptive ramp (ramp_var).** A target parameter is approached in multiplicative steps $p \leftarrow p(1 + \delta)$; after each step the wind is re-relaxed from the current solution. On success the step accepts and δ doubles; on failure the guess is restored, the parameter reset, and δ halves. The run aborts a parameter only after many consecutive failures.
- **Ramp order (ramp_to).** $F_{\text{XUV}} \rightarrow \text{gravity } (R_{\text{pl}}, M_{\text{pl}}) \rightarrow \text{star } (M_*, a, L_*)$, each by ramp_var. The converged solution of one step is the guess for the next (in the port this is an in-memory hand-off; the reference wrapper round-trips it through guess.inp).

The port currently keeps the base BCs fixed at the seed values during the ramp (“static BCs”). This suffices for hot Jupiters near the seed; for planets far from it (very different gravity/irradiation) the base BCs should be re-converged each step — the self-consistent base_bcs/converge_Ncol_sp machinery of the reference wrapper — which is the main deferred item (§7.).

5. The Fortran port

The reference Wind-AE solver core is C (~6.3k lines incl. Numerical Recipes) driven by a Python wrapper that does the continuation. EXHALE’s port reimplements both in Fortran, self-contained under src/modules/wind_ae/ (the only file that touches EXHALE’s globals is the bridge of §6.). Module map:

Module(s)	Role
wae_config, _types, _grid, _status	sizes/constants, derived types, the shared q -grid, the graceful-failure flag
wae_rate_coeffs, _rnew, _rrec, _fe, _ion_pots, _lc_*	auto-generated atomic-data tables (secondary-ionization, recombination, ionization potentials, line-cooling coefficients)
wae_spectrum	GLQ multifrequency spectrum + planet parameters
wae_glq_rates	photoionization/heating rates (incl. the column-density rate cache)
wae_soe	physics: μ , γ , recombination, Q , all the d/dr 's, and the sonic-point linearization
wae_eqns	residual equations + the eval_eqn dispatcher
wae_difeq	numerical-Jacobian assembly of the solvde matrix
wae_nr	the relaxation stack solvde/pinvs/red/bksub
wae_intode	outward Bulirsch–Stoer integration
wae_params, _continuation	input reading; seed load, ramp_var/ramp_to
wae_ic_writer	project the solution onto the EXHALE grid, write *_IC.txt
wae_exhale_input	map an EXHALE input.inp to Wind-AE parameters

Validation (against the C/Python reference). Each layer was gated with a trace-driver comparison: the photoionization/heating rates and the per-species dY_s/dr are *bit-identical*; the heating/cooling Q and the residual equations match to machine precision except where the libm erf differs from gfortran's by ~ 1 ulp (a benign $\sim 10^{-10}$, washed out at the Newton fixed point); the **full windsoln** (relaxation + outward integration) reproduces the C solution to relative $\leq 2 \times 10^{-11}$ on three planets; and the continuation ramp reproduces the Python wrapper's mass-loss rate to five digits. A subtle quirk of the reference — the rate cache keys on column density only and ignores Y_s changes — had to be reproduced exactly for the numerical Jacobian to match.

6. Using it in EXHALE

In-process: IC mode: windae. With this key in input.inp (and Load IC? False), init.f90 calls wae_generate_ic (module wae_exhale_bridge), which maps the planet's EXHALE parameters to Wind-AE ($F_{\text{XUV}} = J_{\text{XUV}}$, $L_\star$ from Stellar Teff/radius, $R_{\text{pl}} = R_0$, the He/H ratio, the base density n_0 and temperature T_0 , and Domain mode $\rightarrow$ tidal on/off), ramps the seed to the planet, solves the wind, and writes the IC onto the EXHALE grid (output/*_IC.txt); load_IC then ingests it. The whole input $\rightarrow$ solve $\rightarrow$ IC $\rightarrow$ run is one EXHALE.x invocation. Because EXHALE re-solves the ionization (and metals) from the first step, the Wind-AE IC is only a *warm start* — finish with Solver: Newton for a quantitative $\dot{M}$.

Standalone: wind_ae_ic.x. make wind_ae_ic builds a separate executable that does the same out of process, reading an input.inp and writing the IC files on a grid taken from an IC_dump.txt; run EXHALE on the result with Load IC? True. The standalone tool shares the solver modules but not the EXHALE bridge, so it stays self-contained.

7. Scope and limitations

- **H/He only.** The port fixes $N_{\text{spec}} = 2$ (H I, He I) — one ionization stage per species. It does *not* model the trace metals, Fe/Ca cooling, the He triplet, or the transit spectrum; those remain EXHALE's job. For an IC this is sufficient: EXHALE adds the metals in equilibrium from the first step, and the IC writer rescales the density to EXHALE's exact He/H.
- **Static-BC continuation.** The ramp keeps the seed's base BCs; it reaches seed-adjacent hot Jupiters well, but strongly-bound or far-from-seed planets (e.g. HD 189733b) ramp slowly or fail. For those, use IC mode: auto. Re-converging the base BCs during the ramp (the base_bcs/ramp_base_bcs/converge_Ncol_sp machinery) is the main planned extension.
- **Substellar geometry, Coriolis cutoff.** Wind-AE is a single substellar streamline integrated only to a few R_{pl} ; beyond the Coriolis turning radius 3-D effects dominate. The mass-loss rate carries the Murray-Clay 0.33 geometric factor when compared to a global rate.
- **erf ~ 1 ulp.** The library error function differs from gfortran's at the last bit; irrelevant for an IC, noted for the record.